

BÁO CÁO BIG DATA

Hệ Thống Phát Hiện Thao Túng Thị Trường

Thành viên: Nguyễn Quang Sang, Nguyễn Đình Phú

Mục lục

1	Tổng Quan Dự Án	3
1.1	Bối cảnh	3
1.2	Mục tiêu	3
2	Kiến trúc hệ thống và dữ liệu lựa chọn	4
2.1	Luồng xử lý tổng quát	4
2.2	Các thành phần chính	5
2.3	Dữ Liệu Và Đặc Trưng	5
2.3.1	Nguồn dữ liệu	5
2.3.2	Chuyển đổi FI-2010 TXT sang replay JSON	5
2.3.3	Chèn spoofing episode để tạo nhãn	6
2.3.4	Tạo feature CSV cho Machine Learning	7
2.3.5	Bộ đặc trưng Machine Learning	7
3	Huấn Luyện Machine Learning	9
3.1	Random Forest bằng scikit-learn	9
3.2	Random Forest bằng Spark MLlib	9
4	Machine Learning Fast Filter	11
4.1	Vai trò trong pipeline	11
4.2	Đặc trưng đầu vào	11
4.3	Liên hệ giữa đặc trưng và spoofing	11
5	Giải thích bài tập	13
5.1	Giải thích thuật toán	13
5.2	Alpha-Beta Search	13
5.3	Hàm đánh giá rủi ro	13
5.4	Đặc tả tìm kiếm Alpha-Beta	13
5.5	Spark RDD và Spark SQL	14
5.6	Phát hiện wash trading	14
5.7	Truy vết tài khoản nghi vấn	15
5.8	Lý do lựa chọn Spark và Redis	15

6	Graph Analytics và Shapley Value	16
6.1	Phát hiện wash trading	16
6.2	Shapley Value	16
7	Kết Quả Demo	17
7.1	Demo tĩnh	17
7.2	Kết luận của demo tĩnh	17
7.3	Demo streaming/replay	17
8	Hướng Dẫn Chạy	20
8.1	Yêu cầu môi trường	20
8.2	Cài thư viện	20
8.3	Build C++ extension	20
8.4	Chạy demo tĩnh	20
8.5	Train Spark ML	20
8.6	Chạy streaming/replay	21
8.7	Luồng kỹ thuật khi chạy demo	21
9	Đánh Giá Và Hạn Chế	22
9.1	Điểm mạnh	22
9.2	Hạn chế	22
9.3	Hướng phát triển	22
10	Kết Luận	23

1. Tổng Quan Dự Án

1.1 Bối cảnh

Thị trường tài chính hiện đại tạo ra lượng dữ liệu số lệnh rất lớn trong thời gian ngắn. Các hành vi như spoofing, layering hoặc wash trading có thể xuất hiện trong vài giây, làm sai lệch tín hiệu cung cầu và gây rủi ro cho nhà đầu tư. Vì vậy, hệ thống giám sát cần vừa xử lý được dữ liệu gần thời gian thực, vừa có khả năng giải thích kết quả để phục vụ phân tích và cung cấp bằng chứng cho các nghiệp vụ kiểm tra đi kèm.

Dự án xây dựng một pipeline phát hiện thao túng thị trường theo hướng lai ghép. C++ được dùng cho lõi Limit Order Book, Python điều phối pipeline, scikit-learn và Spark MLlib huấn luyện bộ lọc Machine Learning, Apache Spark phân tán quá trình phân tích kịch bản, NetworkX phát hiện cộng đồng giao dịch chéo và Shapley Value xác định tài khoản có vai trò trung tâm trong cụm nghi vấn.

Trong phạm vi project, hệ thống tập trung vào hai nhóm hành vi chính. Thứ nhất là spoofing trên dữ liệu số lệnh, tức là hành vi tạo thanh khoản giả bằng cách đặt lệnh lớn rồi hủy nhanh trước khi khớp. Thứ hai là wash trading ở mức quan hệ tài khoản, tức là các tài khoản có liên quan giao dịch qua lại để tạo volume giả. Hai nhóm hành vi này được xử lý bằng hai hướng phân tích khác nhau: spoofing được phát hiện chủ yếu qua đặc trưng Limit Order Book và Machine Learning, còn wash trading được minh họa bằng Graph Analytics và Shapley Value.

1.2 Mục tiêu

- Mô phỏng và cập nhật trạng thái Limit Order Book với hiệu năng cao.
- Sử dụng thông tin số lệnh để phát hiện dấu hiệu spoofing.
- Huấn luyện mô hình Random Forest nhẹ bằng scikit-learn và Spark MLlib để làm một lớp lọc ban đầu.
- Phân tán phân tích nhiều kịch bản rủi ro bằng Spark RDD và tổng hợp bằng Spark SQL.
- Phát hiện cụm wash trading bằng đồ thị có hướng.
- Tính Shapley Value để xác định tài khoản có vai trò trung tâm trong cụm nghi vấn.
- Lưu kết quả chạy thành JSON/CSV để có thể kiểm tra lại từng bước của pipeline.

2. Kiến trúc hệ thống và dữ liệu lựa chọn

2.1 Luồng xử lý tổng quát

```
Replay / Binance Stream
|
v
Redis Stream
|
v
Python Orchestrator
|
+-- Layer 1: ML Fast Filter
+-- Layer 2: C++ Limit Order Book + Alpha-Beta Search
+-- Layer 3: Spark RDD + Spark SQL
+-- Layer 4: Graph Analytics + Shapley Value
|
v
Output: JSON / CSV
```

Luồng xử lý của project được thiết kế theo nhiều tầng. Tầng đầu tiên là Machine Learning Fast Filter, dùng để phát hiện nhanh các trạng thái sổ lệnh có dấu hiệu spoofing. Nếu xác suất spoofing thấp, hệ thống có thể bỏ qua trạng thái đó để tiết kiệm tài nguyên. Nếu trạng thái có rủi ro cao, pipeline tiếp tục chuyển sang tầng phân tích sâu bằng C++ Limit Order Book, Alpha-Beta Search và Spark.

Sau khi có kết quả rủi ro từ nhiều kịch bản, hệ thống tiếp tục xử lý phần wash trading bằng cách mô hình hóa giao dịch tài khoản thành đồ thị có hướng. Các cộng đồng giao dịch chéo được phát hiện bằng strongly connected components, sau đó Shapley Value được sử dụng để tìm tài khoản có đóng góp lớn nhất vào thanh khoản nội bộ của cụm nghi vấn.

2.2 Các thành phần chính

Thành phần	File chính	Vai trò
C++ Limit Order Book	src/cpp/lob.cpp, src/cpp/lob.h	Quản lý bid/ask, thêm lệnh, khớp lệnh, hủy lệnh, tính trạng thái thị trường cũng như các thuật toán duyệt cây.
Python binding	src/cpp/bindings.cpp	Đóng gói C++ thành module Python <code>lob_core</code> bằng <code>pybind11</code> .
ML fast filter	src/python/ml_filter.py	Trích xuất đặc trưng và dự đoán xác suất spoofing.
Train sklearn model	src/python/train_ml_model.py	Huấn luyện Random Forest bằng scikit-learn trên feature CSV.
Train Spark ML model	src/python/train_spark_ml_model.py	Huấn luyện RandomForestClassifier bằng Spark MLlib.
Search engine	src/python/search_engine.py	Phân tích kịch bản bằng Alpha-Beta và heuristic risk score.
Spark engine	src/python/spark_engine.py	Phân tán các kịch bản rủi ro và tổng hợp thống kê.
Replay engine	src/python/replay_manipulation.py	Đẩy dữ liệu replay vào Redis Stream để kiểm thử streaming.
Streaming engine	src/python/streaming_engine.py	Đọc Redis Stream bằng Spark Structured Streaming và phát hiện cảnh báo.
Graph/Shapley	src/python/shapley_analyzer.py	Phát hiện cộng đồng wash trading và tính Shapley Value.
Output recorder	src/python/output_recorder.py	Lưu JSON/CSV theo từng lần chạy.

2.3 Dữ Liệu Và Đặc Trưng

2.3.1 Nguồn dữ liệu

Do dữ liệu sổ lệnh thực tế thường khó tiếp cận hoặc cần trả phí như LOBSTER, bài tập sử dụng FI-2010 làm dữ liệu nền cho Limit Order Book. FI-2010 cung cấp trạng thái sổ lệnh theo thời gian, phù hợp để mô phỏng luồng cập nhật bid/ask và xây dựng dữ liệu huấn luyện cho bài toán phát hiện spoofing.

Tuy nhiên, FI-2010 không có nhãn spoofing rõ ràng và cũng không chứa thông tin định danh tài khoản buyer/seller. Vì vậy, project xử lý dữ liệu theo hai hướng. Thứ nhất, FI-2010 được chuyển sang replay JSON và chèn thêm spoofing episode tổng hợp để tạo dữ liệu có nhãn cho Machine Learning. Thứ hai, phần wash trading được mô phỏng bằng dữ liệu giao dịch giữa các tài khoản để minh họa module Graph Analytics và Shapley Value.

Các luồng dữ liệu chính trong project gồm:

- Dữ liệu FI-2010 dạng TXT ban đầu, dùng làm nền cho Limit Order Book.
- Dữ liệu replay JSON, dùng để mô phỏng luồng cập nhật sổ lệnh và kiểm thử streaming/replay.
- Dữ liệu feature CSV, được sinh từ replay JSON để huấn luyện Machine Learning.
- Dữ liệu giao dịch tài khoản mô phỏng, dùng cho module wash trading và Shapley Value.

2.3.2 Chuyển đổi FI-2010 TXT sang replay JSON

Dữ liệu FI-2010 ban đầu được lưu ở dạng tệp TXT, trong đó các giá trị số biểu diễn trạng thái Limit Order Book tại nhiều thời điểm. Mỗi snapshot của sổ lệnh gồm nhiều mức giá ở hai phía bid và ask. Trong project, nhóm sử dụng 10 level đầu tiên của order book. Mỗi level gồm 4 giá trị:

- `ask_price`: giá bán tại một mức giá phía ask.
- `ask_volume`: khối lượng đang chờ bán tại mức giá đó.
- `bid_price`: giá mua tại một mức giá phía bid.
- `bid_volume`: khối lượng đang chờ mua tại mức giá đó.

Như vậy, 10 level đầu tiên tương ứng với 40 giá trị đầu tiên của mỗi snapshot:

```
Level 1: ask_price_1, ask_volume_1, bid_price_1, bid_volume_1
Level 2: ask_price_2, ask_volume_2, bid_price_2, bid_volume_2
...
Level 10: ask_price_10, ask_volume_10, bid_price_10, bid_volume_10
```

Dữ liệu TXT phù hợp cho xử lý offline nhưng chưa phù hợp với pipeline streaming. Vì vậy, nhóm chuyển từng snapshot sang dạng replay JSON để mô phỏng dữ liệu sự kiện theo thời gian. Mỗi event JSON có dạng:

```
{
  "e": "depthUpdate",
  "E": 1718800000000,
  "b": [["bid_price", "bid_volume"], "..."],
  "a": [["ask_price", "ask_volume"], "..."]
}
```

Trong đó, `e` là loại sự kiện, `E` là timestamp, `b` là danh sách các mức giá phía bid và `a` là danh sách các mức giá phía ask. Mỗi phần tử trong `b` hoặc `a` gồm hai giá trị `[price, volume]`. Cấu trúc này giúp dữ liệu FI-2010 có thể được replay vào Redis Stream và xử lý bằng Spark Structured Streaming giống một luồng cập nhật số lệnh.

Trong quá trình đọc dữ liệu, nếu file FI-2010 có dạng `features × samples`, nhóm chuyển vị ma trận để đưa về dạng `samples × features`, tức là mỗi dòng tương ứng với một snapshot theo thời gian. Sau đó, script chuyển đổi lấy 40 giá trị đầu tiên của mỗi dòng để tạo 10 level bid/ask trong replay event.

2.3.3 Chèn spoofing episode để tạo nhãn

FI-2010 không có nhãn spoofing rõ ràng, trong khi mô hình supervised learning cần dữ liệu có label. Vì vậy, nhóm sử dụng dữ liệu FI-2010 làm nền normal order book và chèn thêm các spoofing episode tổng hợp.

Mặc định, các event ban đầu được gán `label=0`, tương ứng với trạng thái bình thường. Với một số đoạn event được chọn ngẫu nhiên, hệ thống tạo spoofing episode bằng cách tăng mạnh volume ở level gần best bid hoặc best ask, sau đó đưa volume về 0 ở cuối episode để mô phỏng hành vi hủy lệnh. Các event nằm trong episode này được gán `label=1`.

Cách inject này mô phỏng đặc điểm thường gặp của spoofing: xuất hiện một lượng thanh khoản lớn bất thường ở một phía của order book, làm lệch tín hiệu cung cầu, sau đó lệnh bị hủy nhanh trước khi khớp. Nhãn spoofing trong project vì vậy là nhãn tổng hợp phục vụ huấn luyện và kiểm thử, chưa phải nhãn thao túng thật từ sàn giao dịch.

Các file replay chính được sử dụng trong project gồm:

- `data/replay/train_normal_replay.json`: dữ liệu train bình thường.

- `data/replay/train_spoofing_replay.json`: dữ liệu train có chèn spoofing.
- `data/replay/test_normal_replay.json`: dữ liệu test bình thường.
- `data/replay/test_spoofing_replay.json`: dữ liệu test có chèn spoofing, dùng cho demo streaming/replay.

2.3.4 Tạo feature CSV cho Machine Learning

Sau khi có replay JSON, nhóm tiếp tục xây dựng tập đặc trưng dạng CSV để huấn luyện Machine Learning. Mỗi event replay được đưa vào C++ Limit Order Book để cập nhật trạng thái sổ lệnh hiện tại. Từ trạng thái này, hệ thống trích xuất các đặc trưng gồm `spread`, `bid_vol`, `ask_vol`, `imbalance`, `cancel_rate` và `label`.

Luồng tạo feature CSV có thể mô tả như sau:

```

Replay JSON event
|
v
Update C++ Limit Order Book
|
v
Extract market features
|
v
Write feature row to CSV

```

Kết quả của bước này là hai file feature CSV dùng cho huấn luyện và kiểm thử:

- `data/ml/fi2010_train_features.csv`
- `data/ml/fi2010_test_features.csv`

Việc tách train và test được thực hiện trên các file nguồn khác nhau để giảm nguy cơ rò rỉ dữ liệu giữa hai tập. Tập train dùng để học mô hình, còn tập test dùng để đánh giá khả năng phát hiện spoofing trên dữ liệu chưa được dùng trong huấn luyện.

2.3.5 Bộ đặc trưng Machine Learning

Mô hình ML sử dụng cùng một tập đặc trưng:

Đặc trưng	Ý nghĩa
<code>spread</code>	Khoảng cách giữa best ask và best bid.
<code>bid_vol</code>	Đại diện cho thanh khoản phía mua. Trong implementation ML filter hiện tại, giá trị này được lấy theo volume lớn nhất ở một price level sau khi aggregate theo giá để nhạy hơn với spoof wall.
<code>ask_vol</code>	Đại diện cho thanh khoản phía bán. Tương tự <code>bid_vol</code> , giá trị này phản ánh volume nổi bật nhất ở phía ask.
<code>imbalance</code>	Mức mất cân bằng thanh khoản giữa bid và ask. Trong ML filter, hệ thống lấy trị tuyệt đối để tập trung vào độ lệch mạnh hay yếu, không phụ thuộc spoofing xảy ra ở phía bid hay ask.
<code>cancel_rate</code>	Tỷ lệ hủy lệnh, dùng để nhận diện hành vi đặt rồi hủy nhanh.

Các đặc trưng này được chọn vì spoofing thường tạo ra volume lớn bất thường ở một phía của sổ lệnh, làm mất cân bằng cung cầu và thường đi kèm hành vi hủy lệnh nhanh. Do đó, các tín hiệu như volume, imbalance và cancel rate có liên hệ trực tiếp với bản chất của hành vi spoofing.

3. Huấn Luyện Machine Learning

3.1 Random Forest bằng scikit-learn

Bài tập thử nghiệm 2 hướng, với hướng đầu tiên là Random Forest bằng scikit-learn. Mô hình được lưu tại `models/spoofing_rf_model.pkl`. Kết quả thực nghiệm được ghi trong `outputs/ml_fi2010_metrics.json`.

Thông số	Giá trị
Loại mô hình	RandomForestClassifier
Số cây	200
Độ sâu tối đa	10
Class weight	<code>balanced_subsample</code>
Train samples	50,000
Test samples	10,000

Metric	Kết quả
Accuracy	0.8461
Precision	0.1177
Recall	0.8333
Positive F1	0.2063
ROC-AUC	0.9120
Thời gian train	3.9s

Kết quả cho thấy accuracy đạt 0.8461, tuy nhiên chỉ số này chịu ảnh hưởng lớn từ tình trạng mất cân bằng dữ liệu vì số mẫu bình thường nhiều hơn đáng kể so với mẫu spoofing. Do đó, không thể chỉ dựa vào accuracy để đánh giá mô hình. Precision, recall, F1-score và ROC-AUC cần được xem xét cùng nhau.

Precision đạt 0.1177, cho thấy trong các cảnh báo spoofing vẫn còn nhiều false positive. Recall đạt 0.8333, nghĩa là mô hình phát hiện được phần lớn các mẫu spoofing trong tập test. ROC-AUC đạt 0.9120, cho thấy mô hình có khả năng phân tách xác suất giữa normal và spoofing khá tốt. Trong bối cảnh hệ thống giám sát, recall cao có ý nghĩa quan trọng vì bỏ sót hành vi thao túng thường nguy hiểm hơn báo nhầm. Tuy nhiên, precision thấp vẫn là hạn chế cần cải thiện bằng cách bổ sung đặc trưng động, điều chỉnh threshold và sử dụng dữ liệu có nhãn chất lượng hơn.

3.2 Random Forest bằng Spark MLlib

Spark ML được bổ sung để chứng minh khả năng huấn luyện mô hình trên framework Big Data thay vì chỉ dùng scikit-learn local. Script huấn luyện là `src/python/train_spark_`

`ml_model.py`. Mô hình sử dụng `pyspark.ml.classification.RandomForestClassifier`, `VectorAssembler` để tạo cột `features`, và `class_weight` để giảm ảnh hưởng mất cân bằng nhãn.

Lần chạy được ghi trong metrics sử dụng 20 cây, độ sâu tối đa 6 và seed 42.

Thông số	Giá trị
Loại mô hình	Spark MLlib RandomForestClassifier
Số cây	20
Độ sâu tối đa	6
Weight column	<code>class_weight</code>
Train label 0 (bình thường)	48,497
Train label 1 (spoofing)	1,503
Thời gian train	4.57 giây

Metric	Kết quả
Accuracy	0.7090
Positive precision	0.0710
Positive recall	0.9208
Positive F1	0.1319
ROC-AUC	0.9107

Spark MLlib có positive recall đạt 0.9208, nghĩa là mô hình phát hiện được phần lớn mẫu spoofing trong tập test và bỏ sót ít hơn so với mô hình scikit-learn. Tuy nhiên, positive precision thấp hơn vì số lượng false positive lớn hơn. Điều này phản ánh sự đánh đổi giữa khả năng phát hiện nhiều spoofing và số lượng cảnh báo nhầm.

Với bộ dữ liệu thử nghiệm chỉ khoảng 60,000 dòng và 5 đặc trưng, Spark MLlib chịu thêm chi phí khởi tạo Spark session, phân vùng dữ liệu và quản lý job, nên thời gian chạy có thể chậm hơn scikit-learn local. Tuy nhiên, đối với luồng dữ liệu số lệnh có quy mô lớn hơn, cơ chế xử lý song song và phân tán của Spark có thể phát huy lợi thế nhờ khả năng phân vùng dữ liệu ra nhiều worker node và giảm áp lực bộ nhớ trên một máy duy nhất.

4. Machine Learning Fast Filter

4.1 Vai trò trong pipeline

Machine Learning Fast Filter là lớp lọc nhanh đầu tiên trong pipeline. Tầng này không được thiết kế để đưa ra kết luận cuối cùng, mà dùng để xác định trạng thái Limit Order Book nào có dấu hiệu nghi vấn và cần chuyển sang các tầng phân tích sâu hơn như C++ Alpha-Beta Search, Spark scenario analysis và Graph/Shapley.

Cách thiết kế này giúp giảm chi phí tính toán. Nếu tất cả event streaming đều phải chạy phân tích kịch bản bằng Spark và C++ thì hệ thống sẽ tốn nhiều tài nguyên. Vì vậy, Random Forest được dùng như một bộ lọc ban đầu: trạng thái nào có xác suất spoofing thấp thì bỏ qua, trạng thái nào vượt ngưỡng thì mới phân tích sâu.

4.2 Đặc trưng đầu vào

Tầng ML trích xuất 5 đặc trưng: spread, bid volume, ask volume, imbalance và cancellation rate. Các đặc trưng được dùng để dự đoán xác suất spoofing bằng Random Forest.

Với p_1^{ask} là best ask, p_1^{bid} là best bid, V_b là đại diện thanh khoản phía bid và V_a là đại diện thanh khoản phía ask:

$$\text{spread} = p_1^{ask} - p_1^{bid}$$

$$\text{imbalance} = \left| \frac{V_b - V_a}{V_b + V_a} \right|$$

$$\text{cancelrate} = \frac{N_{cancel}}{N_{new} + N_{cancel}}$$

Trong đó, **spread** cho biết khoảng cách giữa giá bán thấp nhất và giá mua cao nhất. **imbalance** cho biết mức độ mất cân bằng thanh khoản giữa hai phía bid và ask. **cancel_rate** phản ánh tỷ lệ lệnh bị hủy trong quá trình cập nhật trạng thái sổ lệnh.

V_b và V_a có thể hiểu là thanh khoản đại diện cho hai phía của sổ lệnh. Trong implementation ML filter hiện tại, **bid_vol** và **ask_vol** được lấy theo volume lớn nhất ở một price level sau khi aggregate theo giá, thay vì tổng toàn bộ các level. Cách này giúp mô hình nhạy hơn với spoof wall, vì spoofing thường tạo một bức tường volume lớn bất thường tại một mức giá cụ thể.

4.3 Liên hệ giữa đặc trưng và spoofing

Spoofing thường có ba dấu hiệu chính trên order book. Thứ nhất, một phía của sổ lệnh xuất hiện volume rất lớn bất thường. Thứ hai, sự xuất hiện của volume này làm mất cân bằng thanh khoản giữa bid và ask. Thứ ba, lệnh lớn thường bị hủy nhanh sau một số event ngắn.

Vì vậy, `bid_vol`, `ask_vol` và `imbalance` giúp mô hình phát hiện sự lệch bất thường về thanh khoản, còn `cancel_rate` giúp phát hiện hành vi đặt rồi hủy nhanh. `spread` được dùng để phản ánh trạng thái giá hiện tại của order book và hỗ trợ nhận diện các trạng thái bất thường như spread quá rộng hoặc crossed book.

Dữ liệu FI-2010 được chuyển sang replay JSON, sau đó các episode spoofing tổng hợp được chèn vào bằng cách tạo cụm lệnh volume lớn gần best bid/best ask và hủy sau một số event ngắn. Nhãn spoofing được gán cho các event nằm trong episode chèn thêm. Để tránh rò rỉ dữ liệu, train và test dùng file nguồn khác nhau, tập test được tạo từ file raw riêng, không lấy lại các cửa sổ replay gần trùng với tập train.

5. Giải thích bài tập

5.1 Giải thích thuật toán

Trong `lob.cpp`, giai đoạn Alpha-Beta Search được mô hình hóa theo hướng tìm kiếm trên cây kịch bản. Sau khi lớp ML filter đánh dấu một trạng thái sở lệnh là nghi vấn, hệ thống sinh ra nhiều hành động giả lập như thêm lệnh lớn, hủy lệnh hoặc giữ nguyên trạng thái để đánh giá tác động lên thị trường. Cách tiếp cận này lấy cảm hứng từ game-theoretic search, trong đó hành vi thao túng và phản ứng của thị trường được xem như các nhánh hành động đối lập. Alpha-Beta Search giúp giảm số nhánh cần duyệt khi đánh giá các kịch bản có rủi ro cao.

5.2 Alpha-Beta Search

Sau khi ML filter đánh dấu một trạng thái có rủi ro, pipeline tạo nhiều kịch bản giả lập quanh trạng thái đó. Search engine đánh giá tác động của các hành động như thêm lệnh lớn, hủy lệnh hoặc giữ nguyên trạng thái. Alpha-Beta Search giúp giảm số nhánh cần duyệt, còn heuristic score đánh giá các yếu tố như spread, imbalance, volume và cancel rate. Quiescence search được dùng để hạn chế việc kết luận nhầm một biến động nhất thời của thị trường thành hành vi thao túng.

5.3 Hàm đánh giá rủi ro

Hàm heuristic kết hợp spread, imbalance và cancellation rate:

$$\text{Risk} = \text{Spread} \times 10 + \text{Imbalance} \times 50 + \text{CancellationRate} \times 30$$

Nếu sở lệnh rơi vào trạng thái crossed book, hệ thống gán điểm rủi ro rất cao để phản ánh bất thường nghiêm trọng.

5.4 Đặc tả tìm kiếm Alpha-Beta

Trong phạm vi project, `alpha_beta_search` được dùng như một lớp heuristic search để đánh giá các trạng thái LOB sinh ra từ kịch bản mô phỏng. Trạng thái s gồm tập lệnh bid/ask hiện tại, best bid, best ask, spread, imbalance và cancellation rate. Hàm tiện ích cuối cùng là:

$$U(s) = \text{Risk}(s)$$

với $\text{Risk}(s)$ được tính bởi công thức heuristic ở trên. Các hành động được xét trong mô phỏng gồm thêm lệnh, hủy lệnh và thực hiện market order với volume thay đổi theo từng scenario. Ở tầng Spark, mỗi scenario được xem là một nhánh độc lập, Alpha-Beta Search chỉ

dùng để lan truyền điểm heuristic trong cây trạng thái nhỏ của từng scenario, không dùng để chứng minh chiến lược tối ưu tuyệt đối của một tác nhân thao túng.

Mô tả rút gọn:

```
alpha_beta(state, depth, alpha, beta, maximizing):
    if depth == 0 or terminal(state):
        return evaluate_state(state)

    '''
    if maximizing:
        value = -inf
        for action in legal_actions(state):
            value = max(value, alpha_beta(apply(state, action),
                                           depth - 1, alpha, beta, false))

            alpha = max(alpha, value)
            if alpha >= beta:
                break
        return value

    value = inf
    for action in legal_actions(state):
        value = min(value, alpha_beta(apply(state, action),
                                       depth - 1, alpha, beta, true))

        beta = min(beta, value)
        if alpha >= beta:
            break
    return value
    '''
```

5.5 Spark RDD và Spark SQL

Các kịch bản được phân tán bằng Spark RDD để chạy song song trên nhiều partition. Mỗi kịch bản trả về một heuristic risk score. Sau đó, kết quả được chuyển thành DataFrame để Spark SQL tổng hợp các thống kê như giá trị trung bình, lớn nhất, nhỏ nhất, độ lệch chuẩn và percentile. Cách làm này tách rõ hai nhiệm vụ: RDD dùng để phân tán tính toán kịch bản, còn Spark SQL dùng để tổng hợp và phân tích kết quả.

Vì các scenario độc lập với nhau, bài toán này phù hợp với cách xử lý song song của Spark. Mỗi partition có thể nhận một nhóm scenario, chạy đánh giá rủi ro và trả về kết quả. Sau đó, driver tổng hợp lại để đưa ra các chỉ số như average risk, maximum risk và percentile. Nếu average risk vượt ngưỡng cảnh báo, hệ thống đánh dấu trạng thái đó là rủi ro cao.

5.6 Phát hiện wash trading

Dự án mô hình hóa giao dịch giữa các tài khoản thành đồ thị có hướng. Mỗi node là một tài khoản, mỗi cạnh là dòng giao dịch từ buyer sang seller. NetworkX tìm các vùng liên thông mạnh để phát hiện nhóm tài khoản có giao dịch chéo lặp đi lặp lại, đây là dấu hiệu thường gặp của wash trading.

5.7 Truy vết tài khoản nghi vấn

Sau khi có cộng đồng nghi vấn, hệ thống dùng Monte Carlo Shapley Value để ước lượng đóng góp của từng tài khoản vào tổng thanh khoản nội bộ. Tài khoản có Shapley Value cao nhất được xem là ứng viên ringleader.

5.8 Lý do lựa chọn Spark và Redis

Đối với thị trường có số lượng cập nhật số lệnh rất lớn, một kiến trúc xử lý tuần tự dễ gặp bottleneck khi phải đánh giá nhiều kịch bản rủi ro. Spark được sử dụng để phân tán quá trình tính toán heuristic và chạy các giả lập kịch bản giao dịch. Sau khi được lọc bởi lớp Random Forest, các trạng thái nghi vấn được chuyển sang lớp phân tích sâu hơn bằng C++ và Spark.

Trong phạm vi project, Redis Stream được chọn làm message buffer nhẹ cho demo streaming vì dễ triển khai trên môi trường local và thao tác đọc/ghi nhanh. Kafka phù hợp hơn với hệ thống production cần log phân tán, khả năng chịu lỗi và mở rộng lớn, nhưng sẽ làm tăng độ phức tạp triển khai trong phạm vi bài tập. Do đó, pipeline streaming hiện tại sử dụng Redis Stream để kết nối replay data với Spark Structured Streaming.

6. Graph Analytics và Shapley Value

6.1 Phát hiện wash trading

Các giao dịch tài khoản được biểu diễn thành đồ thị có hướng bằng NetworkX:

$$G = (V, E, w, t)$$

trong đó V là tập tài khoản, E là giao dịch có hướng từ tài khoản bán sang tài khoản mua, w là volume giao dịch và t là thời điểm hoặc thứ tự event. Hệ thống tìm các strongly connected components để phát hiện cụm giao dịch chéo. Kết quả này chỉ cho thấy các tài khoản có quan hệ giao dịch hai chiều hoặc theo chu trình trong cùng cụm, nó chưa tự chứng minh wash trading, ý định thao túng hoặc quyền kiểm soát chung. Vì vậy, cụm được gắn nhãn là nhóm nghi vấn và cần được kiểm tra cùng volume, tần suất, timing và ngữ cảnh giao dịch.

6.2 Shapley Value

Sau khi có community nghi vấn, Spark RDD phân tán nhiều hoán vị Monte Carlo để ước lượng Shapley Value. Hàm đặc trưng $v(S)$ của một coalition S được hiểu là tổng đóng góp thanh khoản/giao dịch nội bộ mà nhóm S tạo ra trong community. Với một tài khoản i , Shapley Value ước lượng đóng góp biên trung bình của i khi được thêm vào các liên minh khác nhau:

$$\phi_i(v) = \frac{1}{M} \sum_{m=1}^M [v(S_m \cup \{i\}) - v(S_m)]$$

trong đó M là số lượng hoán vị Monte Carlo ngẫu nhiên được chọn, và S_m là tập các tài khoản đứng trước tài khoản i trong hoán vị thứ m .

Shapley Value được dùng vì nó không chỉ đếm số lượng giao dịch hoặc số cạnh của một tài khoản, mà đo đóng góp biên trung bình của tài khoản đó vào volume nội bộ của community. Do đó, kết quả Shapley giúp hệ thống đưa ra một ứng viên ringleader có cơ sở giải thích rõ hơn so với chỉ dùng degree hoặc tổng số giao dịch.

7. Kết Quả Demo

7.1 Demo tĩnh

Pipeline tĩnh lưu kết quả vào thư mục `outputs/<run_id>/`. Một lần chạy gần nhất có `run_id` là 20260621_153840. Các artifact chính gồm:

- `01_initial_lob_snapshot.json`: trạng thái LOB ban đầu.
- `02_ml_filter_result.json`: xác suất spoofing từ ML filter.
- `03_scenario_results.csv`: kết quả từng kịch bản.
- `04_spark_summary.json`: thống kê tổng hợp từ Spark.
- `05_top5_scenarios.json/csv`: top kịch bản rủi ro.
- `06_wash_trading_community.json`: cụm giao dịch chéo nghi vấn.
- `07_shapley_values.json/csv`: giá trị Shapley của các tài khoản.
- `10_final_result.json`: kết luận cuối cùng của pipeline.

7.2 Kết luận của demo tĩnh

Kết quả cuối cùng trong `10_final_result.json`:

Trường	Giá trị
Status	SPOOFING_AND_WASH_TRADING
Severity	CRITICAL
Ringleader	$ACC_{RINGLEADER}$
Community	$ACC_1, ACC_2, ACC_3, ACC_{RINGLEADER}$
Average risk	54.31
Maximum risk	78.91
Spoofing probability	1.0

Kết quả này cho thấy pipeline đã đi hết toàn bộ chuỗi xử lý: tạo trạng thái LOB, chạy ML filter, phân tích kịch bản bằng Spark, phát hiện cộng đồng wash trading và tính Shapley Value để xác định ringleader.

7.3 Demo streaming/replay

Ngoài demo tĩnh, project có demo streaming/replay để kiểm tra luồng xử lý gần thời gian thực. Demo này dùng Redis Stream làm message buffer, Spark Structured Streaming làm engine đọc dữ liệu theo micro-batch và replay engine để đẩy dữ liệu order book vào Redis.

Trong demo chính, nhóm sử dụng file:

```
data/replay/test_spoofing_replay.json
```

File này được chọn vì có chứa spoofing episode tổng hợp và có tính tái lập. Khi replay chạy, từng event trong file JSON được đẩy vào Redis Stream. Spark Streaming đọc các event này, parse danh sách bid/ask, cập nhật trạng thái Limit Order Book, trích xuất đặc trưng và gọi mô hình Random Forest để dự đoán xác suất spoofing. Nếu xác suất vượt ngưỡng, trạng thái được chuyển sang các tầng phân tích sâu hơn.

Luồng demo streaming/replay có thể mô tả như sau:

```
test_spoofing_replay.json
|
v
replay_manipulation.py
|
v
Redis Stream
|
v
streaming_engine.py
|
v
ML Fast Filter + C++ LOB + Spark Analysis
|
v
Alert / Output
```

Demo streaming/replay được sử dụng để kiểm chứng khả năng vận hành của pipeline trên luồng dữ liệu gần thời gian thực. Với dữ liệu replay, hệ thống có thể kiểm soát và tái lập các event spoofing. Bên cạnh đó, nhóm đã chạy thử nghiệm trực tiếp với Binance WebSocket để kiểm tra khả năng tiếp nhận dữ liệu thị trường thật, ghi dữ liệu vào Redis Stream và xử lý bằng Spark Structured Streaming. Hai ảnh terminal dưới đây minh họa cả tốc độ tiếp nhận dữ liệu đầu vào và quá trình hệ thống phát hiện một batch bất thường, sau đó chuyển trạng thái sang tầng C++ và Spark để phân tích sâu hơn.

```

2026-06-25 15:01:30,677 [INFO] --- Batch xử lý hoàn tất ---
2026-06-25 15:01:31,219 [INFO] Received command c on object id p0
2026-06-25 15:01:32,271 [INFO] --- Bắt đầu xử lý Batch ID: 272 (3453 events) ---
2026-06-25 15:01:32,271 [INFO] Nhận Binance depth snapshot. Rebuild LOB từ snapshot mới nhất trong batch.
2026-06-25 15:01:32,271 [INFO] LOB Snapshot -> Best Bid: 0.05002000 | Best Ask: 0.05003000 | Spread: 0.00001000
2026-06-25 15:01:32,279 [INFO] [Feature Extraction] spread=0.0000, bid_vol=4548200.00, ask_vol=4654500.00, imbalance=0.0116, cancel_rate=0.0000
2026-06-25 15:01:40,757 [INFO] Xác suất thao túng (ML): 38.38%
2026-06-25 15:01:40,757 [INFO] Thị trường bình thường.
2026-06-25 15:01:40,757 [INFO] --- Batch xử lý hoàn tất ---
2026-06-25 15:01:41,209 [INFO] Received command c on object id p0
2026-06-25 15:01:41,579 [INFO] --- Bắt đầu xử lý Batch ID: 273 (1279 events) ---
2026-06-25 15:01:41,579 [INFO] Nhận Binance depth snapshot. Rebuild LOB từ snapshot mới nhất trong batch.
2026-06-25 15:01:41,585 [INFO] LOB Snapshot -> Best Bid: 0.05813000 | Best Ask: 0.05814000 | Spread: 0.00001000
2026-06-25 15:01:41,589 [INFO] [Feature Extraction] spread=0.0000, bid_vol=10003800.00, ask_vol=5639000.00, imbalance=0.2790, cancel_rate=0.0000
2026-06-25 15:01:50,766 [INFO] Xác suất thao túng (ML): 66.32%
2026-06-25 15:01:50,766 [WARNING] CẢNH BÁO: ML Filter phát hiện bất thường. Chuyển cho Layer 2 (C++)...
2026-06-25 15:01:50,766 [INFO] [SparkEngine] Broadcast LOB state tới 12 workers...
2026-06-25 15:01:50,775 [INFO] [SparkEngine] Phân phối 200 kịch bản qua Spark RDD (local[*])...
2026-06-25 15:02:08,470 [INFO] [SparkEngine] Hoàn thành trong 8.8783s. Avg Risk: 24.34, Max Risk: 24.37
2026-06-25 15:02:08,470 [WARNING] Spark Distributed Alpha-Beta xong trong 8.878s. Avg Risk: 24.34, Max Risk: 24.37
2026-06-25 15:02:08,470 [INFO] --- Batch xử lý hoàn tất ---
2026-06-25 15:02:09,044 [INFO] Received command c on object id p0
2026-06-25 15:02:09,916 [INFO] --- Bắt đầu xử lý Batch ID: 274 (3991 events) ---
2026-06-25 15:02:09,922 [INFO] Nhận Binance depth snapshot. Rebuild LOB từ snapshot mới nhất trong batch.
2026-06-25 15:02:09,922 [INFO] LOB Snapshot -> Best Bid: 7.53100000 | Best Ask: 7.53200000 | Spread: 0.00100000
2026-06-25 15:02:09,935 [INFO] [Feature Extraction] spread=0.0010, bid_vol=197481.00, ask_vol=272508.00, imbalance=0.1596, cancel_rate=0.0000
2026-06-25 15:02:19,478 [INFO] Xác suất thao túng (ML): 14.77%
2026-06-25 15:02:19,478 [INFO] Thị trường bình thường.
2026-06-25 15:02:19,478 [INFO] --- Batch xử lý hoàn tất ---
2026-06-25 15:02:20,054 [INFO] Received command c on object id p0
2026-06-25 15:02:20,527 [INFO] --- Bắt đầu xử lý Batch ID: 275 (1948 events) ---
2026-06-25 15:02:20,527 [INFO] Nhận Binance depth snapshot. Rebuild LOB từ snapshot mới nhất trong batch.
2026-06-25 15:02:20,532 [INFO] LOB Snapshot -> Best Bid: 61907.99000000 | Best Ask: 61908.00000000 | Spread: 0.01000000
2026-06-25 15:02:20,532 [INFO] [Feature Extraction] spread=0.0100, bid_vol=424.00, ask_vol=62.00, imbalance=0.7449, cancel_rate=0.0000
2026-06-25 15:02:29,841 [INFO] Xác suất thao túng (ML): 56.93%
2026-06-25 15:02:29,841 [INFO] Thị trường bình thường.

```

Hình 7.1: Spark Structured Streaming xử lý dữ liệu Binance theo micro-batch. Batch 273 được mô hình ML đánh giá có xác suất thao túng 66.32%, sau đó được chuyển sang tầng C++ và Spark Distributed Alpha-Beta để phân tích.

```

2026-06-25 15:01:19,222 - BinanceWS - INFO - [Binance input] messages=259000, depth_updates=258987, input=320.05 MB (0.31255296 GB), output=347.58 MB (0.33943240 GB), avg_msg=1.27 KB
, rate=91.47 msg/s, bandwidth=115.75 KB/s
2026-06-25 15:01:26,900 - BinanceWS - INFO - [Binance input] messages=260000, depth_updates=259987, input=321.29 MB (0.31376120 GB), output=348.92 MB (0.34074440 GB), avg_msg=1.27 KB
, rate=91.58 msg/s, bandwidth=115.88 KB/s
2026-06-25 15:01:36,936 - BinanceWS - INFO - [Binance input] messages=260917, depth_updates=260904, input=322.43 MB (0.31486991 GB), output=350.16 MB (0.34194827 GB), avg_msg=1.27 KB
, rate=91.57 msg/s, bandwidth=115.88 KB/s
2026-06-25 15:01:37,294 - BinanceWS - INFO - [Binance input] messages=261000, depth_updates=260987, input=322.53 MB (0.31497031 GB), output=350.27 MB (0.34205730 GB), avg_msg=1.27 KB
, rate=91.59 msg/s, bandwidth=115.90 KB/s
2026-06-25 15:01:42,645 - BinanceWS - INFO - [Binance input] messages=262000, depth_updates=261987, input=323.77 MB (0.31617833 GB), output=351.61 MB (0.34336903 GB), avg_msg=1.27 KB
, rate=91.77 msg/s, bandwidth=116.13 KB/s
2026-06-25 15:01:48,647 - BinanceWS - INFO - [Binance input] messages=263000, depth_updates=262987, input=325.00 MB (0.31738614 GB), output=352.95 MB (0.34468064 GB), avg_msg=1.27 KB
, rate=91.93 msg/s, bandwidth=116.33 KB/s
2026-06-25 15:01:53,360 - BinanceWS - INFO - [Binance input] messages=264000, depth_updates=263987, input=326.24 MB (0.31859212 GB), output=354.29 MB (0.34599040 GB), avg_msg=1.27 KB
, rate=92.13 msg/s, bandwidth=116.58 KB/s
2026-06-25 15:02:01,619 - BinanceWS - INFO - [Binance input] messages=265000, depth_updates=264987, input=327.47 MB (0.31979978 GB), output=355.64 MB (0.34730182 GB), avg_msg=1.27 KB
, rate=92.21 msg/s, bandwidth=116.68 KB/s
2026-06-25 15:02:11,672 - BinanceWS - INFO - [Binance input] messages=265890, depth_updates=265877, input=328.57 MB (0.32087401 GB), output=356.83 MB (0.34846841 GB), avg_msg=1.27 KB
, rate=92.20 msg/s, bandwidth=116.67 KB/s
2026-06-25 15:02:12,301 - BinanceWS - INFO - [Binance input] messages=266000, depth_updates=265987, input=328.71 MB (0.32100703 GB), output=356.98 MB (0.34861285 GB), avg_msg=1.27 KB
, rate=92.21 msg/s, bandwidth=116.69 KB/s
2026-06-25 15:02:16,616 - BinanceWS - INFO - [Binance input] messages=267000, depth_updates=266987, input=329.95 MB (0.32221507 GB), output=358.32 MB (0.34992473 GB), avg_msg=1.27 KB
, rate=92.42 msg/s, bandwidth=116.95 KB/s
2026-06-25 15:02:21,301 - BinanceWS - INFO - [Binance input] messages=268000, depth_updates=267987, input=331.19 MB (0.32342286 GB), output=359.67 MB (0.35123641 GB), avg_msg=1.27 KB
, rate=92.62 msg/s, bandwidth=117.20 KB/s
2026-06-25 15:02:27,298 - BinanceWS - INFO - [Binance input] messages=269000, depth_updates=268987, input=332.42 MB (0.32462914 GB), output=361.01 MB (0.35254649 GB), avg_msg=1.27 KB
, rate=92.77 msg/s, bandwidth=117.40 KB/s
2026-06-25 15:02:35,450 - BinanceWS - INFO - [Binance input] messages=270000, depth_updates=269987, input=333.66 MB (0.32583511 GB), output=362.35 MB (0.35385628 GB), avg_msg=1.27 KB
, rate=92.86 msg/s, bandwidth=117.50 KB/s
2026-06-25 15:02:45,478 - BinanceWS - INFO - [Binance input] messages=270902, depth_updates=270889, input=334.77 MB (0.32692163 GB), output=363.56 MB (0.35503642 GB), avg_msg=1.27 KB
, rate=92.85 msg/s, bandwidth=117.49 KB/s
2026-06-25 15:02:52,355 - BinanceWS - INFO - [Binance input] messages=271000, depth_updates=270987, input=334.89 MB (0.32704027 GB), output=363.69 MB (0.35516522 GB), avg_msg=1.27 KB
, rate=92.66 msg/s, bandwidth=117.25 KB/s
2026-06-25 15:03:02,450 - BinanceWS - INFO - [Binance input] messages=271295, depth_updates=271282, input=335.25 MB (0.32739610 GB), output=364.08 MB (0.35555168 GB), avg_msg=1.27 KB
, rate=92.44 msg/s, bandwidth=116.98 KB/s
2026-06-25 15:03:12,628 - BinanceWS - INFO - [Binance input] messages=271370, depth_updates=271357, input=335.35 MB (0.32748668 GB), output=364.19 MB (0.35565003 GB), avg_msg=1.27 KB
, rate=92.15 msg/s, bandwidth=116.61 KB/s

```

Hình 7.2: Thống kê dữ liệu đầu vào từ Binance WebSocket trên terminal, gồm tổng số message, dung lượng dữ liệu, kích thước message trung bình, tốc độ nhận và băng thông.

8. Hướng Dẫn Chạy

8.1 Yêu cầu môi trường

Trên Windows:

- Python 3.11 64-bit.
- Java 17 cho PySpark.
- CMake và MSVC Build Tools để build C++ extension.
- Redis hoặc Docker Desktop nếu chạy streaming/replay.

8.2 Cài thư viện

Trên Windows:

```
cd path\to\BigDataSang
py -3.11 -m pip install -r requirements.txt
```

8.3 Build C++ extension

Python không import trực tiếp file C++, cần build lỗi C++ thành extension Python tương thích với môi trường chạy.

Trên Windows:

```
cd path\to\BigDataSang
$env:PYTHON_EXE=(py -3.11 -c "import sys; print(sys.executable)")
cmake -S . -B build-py311-v2 -G "NMake Makefiles" -DCMAKE_POLICY_VERSION_MINIMUM=3.5 -
DPYTHON_EXECUTABLE="$env:PYTHON_EXE"
cmake --build build-py311-v2
```

8.4 Chạy demo tĩnh

Trên Windows:

```
cd path\to\BigDataSang
$env:PYTHONIOENCODING="utf-8"
$env:PYSARK_PYTHON=(py -3.11 -c "import sys; print(sys.executable)")
$env:PYSARK_DRIVER_PYTHON=$env:PYSARK_PYTHON
& $env:PYSARK_PYTHON src/python/main.py
```

8.5 Train Spark ML

Trên Windows:

```
cd path\to\BigDataSang
py -3.11 src/python/train_spark_ml_model.py --num-trees 20 --max-depth 6
```

Sau khi chạy, kiểm tra kết quả tại `outputs/spark_ml_fi2010_metrics.json` và `models/spark_ml_rf/`.

8.6 Chạy streaming/replay

Bước 1: Khởi động Redis (Có thể dùng Docker hoặc dịch vụ Redis chạy trực tiếp trên host):

```
docker compose up -d
```

Bước 2: Khởi động Spark streaming engine (Terminal 1)

Trên Windows:

```
cd path\to\BigDataSang
$env:HADOOP_HOME="$PWD\hadoop"
$env:hadoop_home=$env:HADOOP_HOME
$env:Path="$env:HADOOP_HOME\bin;$env:Path"
$env:PYTHONIOENCODING="utf-8"
$env:PYSARK_PYTHON=(py -3.11 -c "import sys; print(sys.executable)")
$env:PYSARK_DRIVER_PYTHON=$env:PYSARK_PYTHON
& $env:PYSARK_PYTHON src/python/streaming_engine.py
```

Bước 3: Đẩy replay data vào Redis Stream (Terminal 2)

Trên Windows:

```
cd path\to\BigDataSang
py -3.11 src/python/replay_manipulation.py --file data/replay/test_spoofing_replay.json --sleep 0.1
```

8.7 Luồng kỹ thuật khi chạy demo

Project hỗ trợ hai chế độ demo chính: demo tĩnh và demo streaming/replay.

Demo tĩnh được chạy bằng `src/python/main.py`. Chế độ này tạo dữ liệu mô phỏng trong nội bộ chương trình và chạy lần lượt các tầng xử lý gồm ML Fast Filter, C++ Limit Order Book, Spark scenario analysis, Graph Analytics và Shapley Value.

Demo streaming/replay dùng ba tiến trình chính. Tiến trình đầu tiên là Redis, đóng vai trò message buffer. Tiến trình thứ hai là `streaming_engine.py`, dùng Spark Structured Streaming để đọc dữ liệu từ Redis Stream theo từng micro-batch. Tiến trình thứ ba là `replay_manipulation.py`, dùng để đọc file replay JSON và đẩy từng event vào Redis Stream.

Trong quá trình streaming, mỗi event replay được parse để lấy timestamp, danh sách bid và danh sách ask. Sau đó, trạng thái Limit Order Book được cập nhật, các đặc trưng như spread, bid volume, ask volume, imbalance và cancel rate được trích xuất, rồi mô hình Random Forest dự đoán xác suất spoofing. Nếu xác suất vượt ngưỡng cảnh báo, hệ thống tiếp tục chạy phân tích kịch bản bằng C++ và Spark để tính risk score.

9. Đánh Giá Và Hạn Chế

9.1 Điểm mạnh

- Kiến trúc lai ghép đầy đủ: C++, Python, Spark, ML, Graph và Shapley Value.
- Có cả demo tĩnh và streaming/replay.
- Kết quả chạy được lưu thành file để kiểm chứng.
- Spark MLlib đã được chạy thành công và sinh model/metrics.
- Luồng dữ liệu được tổ chức thành các bước rõ ràng: FI-2010 TXT, replay JSON, feature CSV, huấn luyện ML và demo streaming.

9.2 Hạn chế

- Dataset mất cân bằng mạnh, nên accuracy không đủ để đánh giá riêng lớp spoofing.
- Nhân spoofing được tạo bằng cách inject episode tổng hợp trên nền FI-2010, chưa phải nhân thao túng thật từ sàn giao dịch.
- Dữ liệu wash trading trong project là dữ liệu tài khoản mô phỏng, vì FI-2010 không có thông tin định danh buyer/seller.
- Các hành vi spoofing và wash trading trong thị trường thực tế rất phức tạp, nên hệ thống hiện mới phù hợp ở mức mô phỏng và demo học thuật, chưa đủ điều kiện áp dụng trực tiếp vào thực tiễn.
- Spark phù hợp với batch hoặc near real-time, nhưng chưa phải lựa chọn tối ưu cho các hệ thống pre-trade latency cực thấp.
- Mô hình hiện được huấn luyện trên dữ liệu cố định, nên chưa có cơ chế cập nhật liên tục để thích nghi với các hành vi thao túng mới.

9.3 Hướng phát triển

- Bổ sung các đặc trưng động theo thời gian như rolling imbalance, rolling cancel rate và biến động spread để Random Forest filter phản ánh tốt hơn trạng thái thị trường.
- Tìm kiếm thêm dữ liệu thực tế hoặc benchmark có chất lượng cao để mô hình học được nhiều pattern thao túng hơn.
- Bổ sung dữ liệu account-level thực tế nếu có điều kiện để đánh giá tốt hơn phần wash trading.
- Điều chỉnh threshold và bổ sung các kỹ thuật xử lý mất cân bằng dữ liệu để giảm false positive.
- Xây dựng dashboard hoặc báo cáo tự động từ các output để hỗ trợ theo dõi kết quả demo.
- Mở rộng luồng streaming từ Redis sang Kafka nếu triển khai theo hướng production.

10. Kết Luận

Dự án đã hoàn thành một pipeline phát hiện thao túng thị trường theo hướng lai ghép. Pipeline bắt đầu từ dữ liệu sổ lệnh FI-2010, chuyển đổi sang replay JSON, chèn spoofing episode để tạo nhãn, trích xuất đặc trưng và huấn luyện Random Forest bằng cả scikit-learn và Spark MLlib. Sau tầng Machine Learning Fast Filter, các trạng thái nghi vấn tiếp tục được phân tích bằng C++ Limit Order Book, Alpha-Beta Search, Spark RDD và Spark SQL. Cuối cùng, Graph Analytics và Shapley Value được sử dụng để minh họa phát hiện wash trading và truy vết tài khoản có vai trò trung tâm.

Kết quả thực nghiệm cho thấy mô hình ML có khả năng phát hiện phần lớn các mẫu spoofing trong tập test, nhưng precision còn thấp do dữ liệu mất cân bằng và nhãn spoofing được tạo tổng hợp. Đây là hạn chế cần cải thiện trong các phiên bản tiếp theo. Tuy vậy, project đã thể hiện được đầy đủ các thành phần của một pipeline Big Data: xử lý dữ liệu, huấn luyện Machine Learning, streaming/replay, phân tán tính toán bằng Spark, lưu output và giải thích kết quả bằng graph-based analysis.